

COMP90025 Project 2

Spell Checker – Checking words

Ziyu Wang (1560831)

1 Introduction

Spelling checkers are now an inseparable part of everyday digital communication. In this project, we explore how distributed computing can improve the efficiency of spell checking by using multiple processors. The system takes a dictionary file and a list of words as inputs, then divides the dictionary into separate portions and broadcast the unchecked word list across all processors. Each processor verify word presence and to generate candidate corrections independently. Together, the combined outputs form a complete list of misspelled words with their correction candidates. The overall process follows five main stages shown in Algorithm 1.

Algorithm 1: Spellcheck

1. Dictionary Slice (see Algorithm 2)
 2. Broadcast Word List
 3. BK-Tree Construction: generate candidate corrections
 4. Get misspelled
 - Present phase
 - Candidate phase
 5. Package
-

Dictionary Slice. Algorithm 2 describes how each MPI process reads its assigned portion of the dictionary file. To allow the file can be accessed concurrently, we MPI_File_read_at with offset. To prevent cutting words across slice boundaries, we also add a 256 byte padding region (PAD). Then, the valid region of each process would lie in the nearest newlines around its offset. This method both prevents segment and keeps dictionary load balance across ranks.

On the other side, the complete list of unchecked words is broadcast from the root process to all ranks using MPI_Bcast. Each rank then structures a local BK-tree to support candidate generation in the following searching phase.

Get misspelled. In this stage, we separate presence checking from candidate generation and apply parallel in thread level. To be more specific, in the **Present phase**, each rank scans the word list to identify misspelled words, and global presence vector combines the results using logic OR (MPI_Allreduce). Only words marked as misspelled would be proceed in the **Candidate phase**. Each rank computes candidate corrections by its BK-tree and performs local sorting and deduplication in parallel. For the parallel strategy,

since presence checking is lightweight, we use static schedule while apply guided schedule with chunk size 64 in second phase since candidate generation varies significantly.

Initially, we tried a single design that mixed presence checking and candidate generation, but experiments showed candidate computation dominated runtime. However, this two-phase approach were proposed to eliminate redundant computation and thus improves overall efficiency. Mathematically, given the number of misspelled words is N and correct word is M , this strategy reduces $O(N \times \text{Candidates})$ from $O((N+M) \times \text{Candidates})$.

Packaging. Finally, each rank packs different length candidate lists into a contiguous buffer using MPI_Gatherv. The root process merges all candidate lists, sorts them first by the number of correction options and then lexicographically, and writes the final results to the output file.

2 Methodology

In this section, we would go through the experimental setting, correctness measurement, and finally the performance evaluation methods.

2.1 Experimental Setup

All experiments were conducted on the Spartan HPC system to ensure consistent hardware conditions and fair evaluation across strategies. We specified the sapphire partition and compiled with GCC/13.3.0 and OpenMPI/5.0.3.

To eliminate inter-node latency effects and maintain reproducible measurements, all experiments were performed on a single node while varying the number of MPI processes (np) and OpenMP threads within that node (omp). Both were configured in powers of two ranging from 1 to 32. This resulted in a total of 21 combinations due to the available physical cores constrain ($np \times omp < 32$).

Three dataset sizes were evaluated:

1. **Small:** dict10000 dictionary with words10000 list.
2. **Medium:** dict100000 dictionary with words100000 list.
3. **Large:** dict626623 dictionary with words658976 list.

2.2 Correctness

We first validated the implementation using a very small test dataset (dictionary.spec.txt and words.spec.txt). Then, we serve the baseline implementation provided in the project skeleton as the gold standard reference in correction testing. The correctness requires the hash values in the output to match the gold reference.

Algorithm 2: ReadSliceAligned

Input : Dictionary file D ; slice offset and length (off, len); file size F ; padding size PAD

read_off $\leftarrow \max(0, \text{off} - \text{PAD})$
max_len $\leftarrow \min(F - \text{read_off}, \text{len} + 2 \times \text{PAD})$
buf $\leftarrow \text{MPI_File_read_at}(D, \text{read_off}, \text{max_len})$
if off = 0 **then**
| begin $\leftarrow 0$
end
else
| begin $\leftarrow$ position of the first newline '\n' at or after (off - read_off - 1) in buf
| begin $\leftarrow \text{begin} + 1$
end
if off + len = F **then**
| end $\leftarrow |\text{buf}|$
end
else
| end $\leftarrow$ position of the last newline '\n' in buf
end
aligned $\leftarrow \text{buf}[\text{begin} : \text{end}]$
base $\leftarrow \text{read_off} + \text{begin}$
lines $\leftarrow \text{emptyset}$
start $\leftarrow 0$
for $j = 0$ **to** |aligned| - 1 **do**
| **if** aligned[j] = '\n' **then**
| | line_start $\leftarrow \text{base} + \text{start}$
| | **if** off $\leq \text{line_start} < \text{off} + \text{len}$ **then**
| | | append substring aligned[start : j] to lines
| | **end**
| | start $\leftarrow j + 1$
| **end**
end
return lines

2.3 Performance Measures

Performance was evaluated in the following four dimensions:

1. Speedup and Efficiency
2. Execution Time Breakdown
3. Dictionary Load Balance
4. Memory Usage

Execution Time Measurement. Execution time was measured based on wall-clock time, started right after MPI_Init and stopped before MPI_Finalize. Individual timers were placed at the start and end of each major phase.

The measured stages include:

- Load dictionary file: Each process reads assigned dictionary.

- Build WordList: The root process reads input word list and broadcasts it to all ranks.
- Construct Dictionary object: The local dictionary text is converted into an internal lookup object.
- Build BK-Tree: Each rank builds BK-tree.
- Get misspelled: All correction candidates are generated.
- Save output: The root merges, sorts, and writes the final results.

The overall wall time (TOTAL) was recorded from the beginning of dictionary loading to the end of output writing.

Speedup and Efficiency. Given

- T_1 : the execution time of the baseline (np=1 and omp=1)
- T_p : the execution time using p total cores (np $\times$ omp)

The **speedup** S_p is defined as:

$$S_p = \frac{T_1}{T_p} \quad (1)$$

and **efficiency** E_p is:

$$E_p = \frac{S_p}{p} \quad (2)$$

An ideal speedup is linear, which means $S_p = p$ and $E_p = 1.0$. However, there is inevitable to have inherently unparallel parts. Therefore, we use Amdahl's Law to calculate theoretical maximum speedup. That is give the the serial fraction f of the program:

$$S_p^{\text{Amdahl}} = \frac{1}{f + \frac{1-f}{p}} \quad (3)$$

When the p approaches to infinity, the speed up would be closed to $\frac{1}{f}$.

Dictionary Load Balance. The dictionary size is the the number of unique words. Ideally, slicing should assign disjoint and equally sized portions of the dictionary to each rank. However, small overlaps are inevitable due to the PAD and newline at slice boundaries in implementation.

Let $|D_i|$ denote the dictionary size of rank i . We define **Duplication Ratio** as:

$$\text{DupRatio} = \frac{\sum_{i=0}^{p-1} |D_i|}{\left| \bigcup_{i=0}^{p-1} D_i \right|} \approx \frac{\sum_i |D_i|}{\text{global_unique}}$$

, where global_unique is the dictionary count from the baseline. A ratio of 1 means there is no cross-rank overlap while values larger than 1 indicate some words duplicated across ranks.

The **Load Balance** is also reported to examine the variance of dictionary size. Since the number of ranks is relatively small, we define a more intuitive imbalance metric as:

$$\text{Imbalance} = \frac{\max_i |D_i| - \min_i |D_i|}{\frac{1}{p} \sum_i |D_i|}.$$

A higher value suggests there is a rank handles the most work and a other rank has less portion of the dictionary.

Memory Usage. Peak memory was measured for individual process using the maximum resident set size (RSS) reported by `/usr/bin/time`.

3 Experiments

This section reports all the experiments results in the following order:

1. Correctness
2. Speedup and Efficiency
3. Execution Time Breakdown
4. Dictionary Load Balance
5. Memory Usage

3.1 Correctness

The below shows result of a small validation dataset.

appel: Rappel appeal rappel

tim: Tim Vim time vim

stap: sap snap stab step stop strap tap

Table 1 summarizes the overall correctness results.

Dataset	Total Runs	Correct	Ratio
Small	21	21	1.00
Medium	21	21	1.00
Large	21	21	1.00

Table 1. Correctness

3.2 Speedup and Efficiency

Figure 8 shows the complete heatmaps of speedup and efficiency across all 21 configurations. However, since the effect of varying the number of threads on runtime and memory consumption is minimal, the following discussion focuses on how performance scales with the number of processes for clearer evaluation.

Table 2 summarizes the speedup values relative to the sequential baseline while Table 3 present the relative speedup when using $np = 1$ as the baseline. Figure 1 further visualizes the outcomes with ideal linear speedup line. Moreover, the corresponding parallel efficiency for each configuration is shown in Table 4.

np	Small	Medium	Large
1	0.9433	0.7293	0.9863
2	1.8320	1.2317	1.4864
4	2.9588	2.3603	2.4467
8	4.9831	5.7315	4.4986
16	8.3267	14.1931	10.0114
32	13.8935	19.1994	21.9092

Table 2. Speedup (Baseline: Sequential)

np	Small	Medium	Large
1	1.0000	1.0000	1.0000
2	1.9381	1.6787	1.4433
4	3.0870	3.3961	2.5630
8	5.1961	7.7240	4.2528
16	8.7633	19.5274	10.7495
32	14.5183	26.0633	21.7558

Table 3. Speedup (Baseline: np=1)

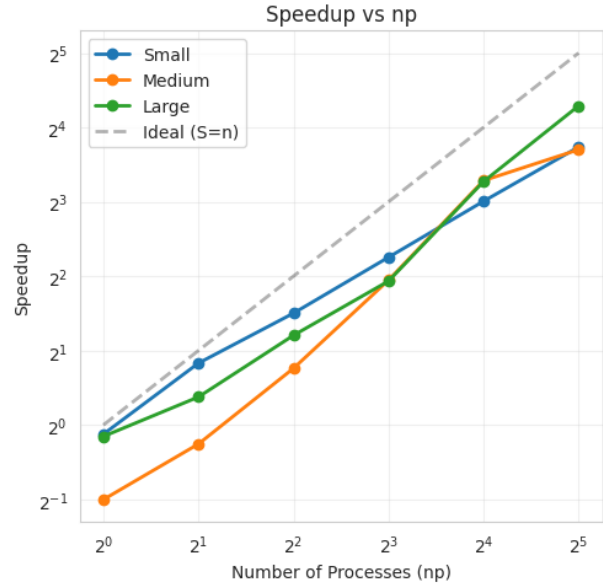


Figure 1. Speedup (Baseline: np=1)

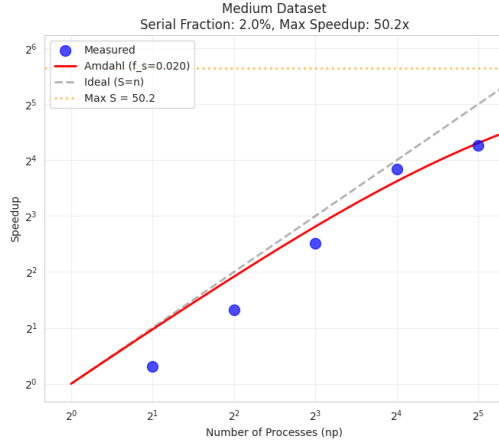
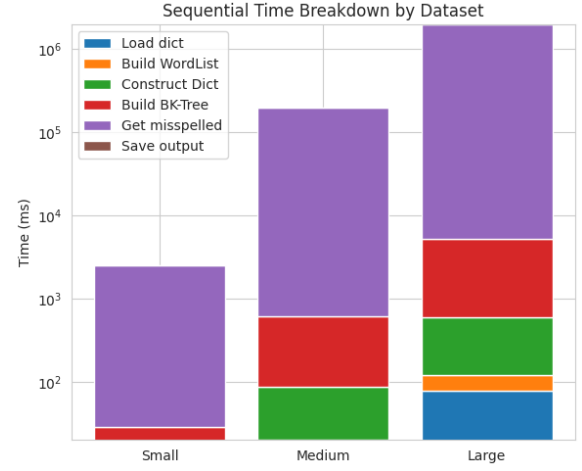
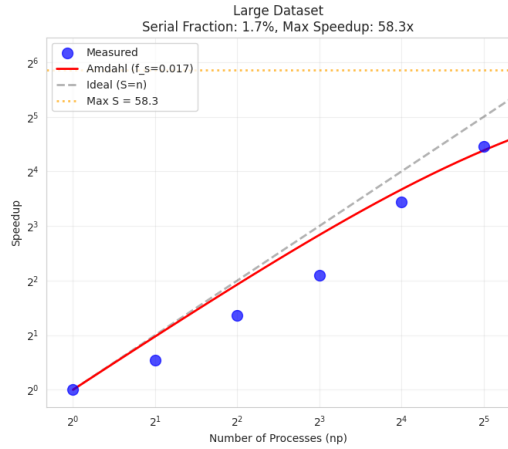
np	Small	Medium	Large
1	1.0000	1.0000	1.0000
2	0.9690	0.8394	0.7217
4	0.7717	0.8490	0.6408
8	0.6495	0.9655	0.5316
16	0.5477	1.2205	0.6718
32	0.4537	0.8145	0.6799

Table 4. Efficiency

Table 5 shows the maximum speedup using Amdahl's Law. With sequential baseline, the **Serial** fraction was estimated by fitting the speedup results to 3 through nonlinear least squares. The **Theoretical Max Speedup** ($1/f_s$) then was calculated using this estimated serial fraction. The **Utilization** was calculated by the actual speedup at 32 processes to show how much of the theoretical upper bound was achieved. Figure 2 and 3 further plot the fitted Amdahl curves with the actual speedups and the asymptotic limit for comparison for medium and large datasets.

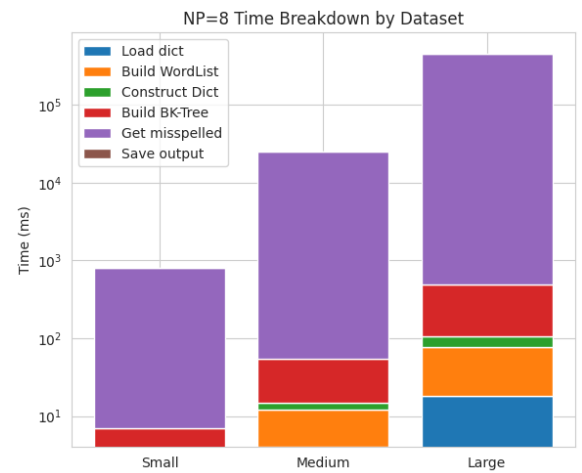
Table 5. Amdahl's Law fitting results

Dataset	Serial	Parallel	Theoretical Max Speedup	np=32 Speedup	Utilization
Small	0.0463	0.9537	21.58×	13.89×	64.4%
Medium	0.0199	0.9801	50.21×	19.20×	38.2%
Large	0.0172	0.9828	58.30×	21.91×	37.6%

**Figure 2.** Amdahl's Law (Medium)**Figure 4.** np=1 runtime breakdown**Figure 3.** Amdahl's Law (Large)

3.3 Execution Time Breakdown

The following figures (Figure 4, 5, 6) break down the run-time to main six computation phases for the 1 processes, 8 processes, and 32 processes cases, respectively.

**Figure 5.** np=8 runtime breakdown

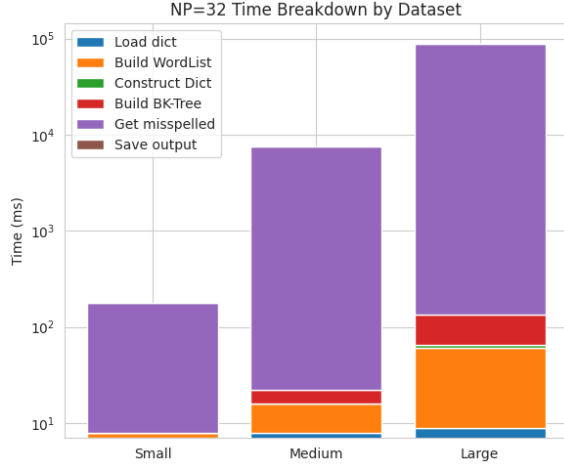


Figure 6. np=32 runtime breakdown

Table 6 reports the dictionary duplication ratio and load imbalance for different process counts using large datasets.

Table 6. Duplication ratio and load imbalance

np	Duplication Ratio	Load Imbalance
2	1.001055	0.007901
4	1.001640	0.067957
8	1.001852	0.103009
16	1.002048	0.187014
32	1.002091	0.309170

Figure 7 plots the execution time against memory usage across different processes and datasets. This figure aims to illustrate the tradeoff between speedup and required memory.

4 Discussion

4.1 Correctness

All three datasets achieve a correctness ratio of 1.00. This outcome confirms the implementation of concurrent reading *Dictionary Slice* does not introduce any race condition and words are not unintentionally cut off at boundaries by adding a padding region. The current padding size is set to 256 bytes and works well for the tested dictionaries while if the dictionary contains longer words, the size should be increased accordingly. On the other side, the size could be reduced to save memory when the maximum word length is confirmed smaller precisely.

4.2 Performance

Moreover, the slicing strategy also benefits performance by keeping low duplicated dictionary. As shown in Table 6, the duplication ratios are inevitably slightly above 1.0 but within

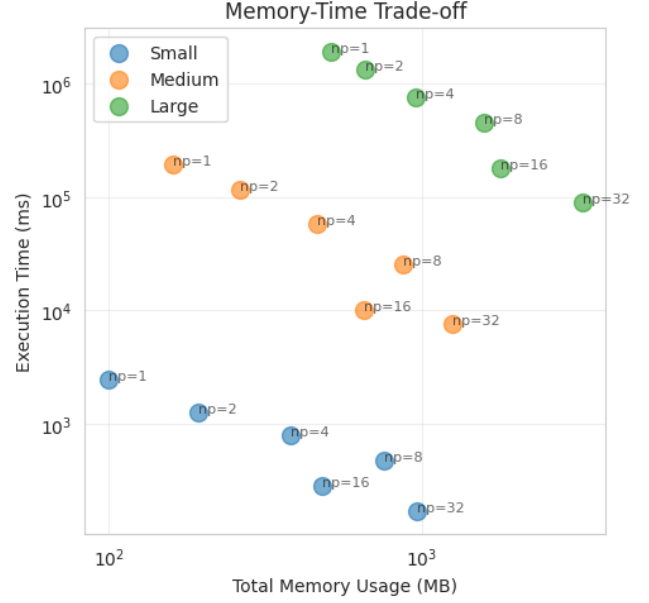


Figure 7. Memory-Speed Tradeoff

0.3%, suggesting there is no significant overlap of dictionary words across ranks. However, the data shows the load imbalance increases with the number of processes mainly because the dictionary is partitioned by byte offset instead of line count. When the number of processes increases and each slice becomes smaller, the relative impact of boundary alignment and variation in word length becomes more significant. Specifically, each process reads a slice starting at an offset and then shifts its boundary to the nearest newline to avoid cutting words. This boundary adjustment could present a noticeable percentage difference. Furthermore, due to the byte slicing, the areas containing longer words result in fewer lines while areas with shorter words contain more. As a result, even though duplication remains low, the number of unique words assigned to each rank becomes uneven, leading to varying workloads among processes. From another perspective, this approach helps to maintain stable memory usage.

Speedup and Efficiency. For the sequential baseline, the single processor performance is near $1.0\times$ for both the small and large datasets but only reaches $0.73\times$ for the medium. This indicates that a MPI initialization overheads, which is hard to be observed in the other datasets, because the workload is either large enough to fully amortize the cost or small enough to finish instantly.

For the single processor as baseline, speedup improves steadily across all datasets. At np=32, the system achieves approximately $15\times$, $26\times$, and $22\times$ for small, medium, and large datasets, respectively. From the efficiency perspective, the medium dataset remains above 80% overall and achieves 1.22% at np=16. This the greatest percentage suggests near

optimal balance between computation and communication while the efficiency begins to drop due to the increasing costs of MPI creation and synchronization.

Runtime Breakdown. Figure 4 shows that *Get misspelled* accounts for the majority of total runtime across all datasets. In addition, dictionary-related stages, including *Load dict*, *Construct Dict*, and *Build BK-Tree*, increase noticeable portion of the total computation as the dataset size grows, which indicates that both dictionary loading and structure construction would scale with input size.

When comparing the three figures, the proportion of *Build WordList* gradually increases as the number of processes grows. This phenomena is expected since only dictionary-related operations are parallelized while the word list still needs to be broadcast from the root process to all ranks. As other stages speed up with more processes, the relative cost of broadcasting becomes more significant, implying that *Build WordList* is limited by communication. This is also observed in Table 7, the *Build WordList* at $np=8$ shows slightly lower performance than at $np=32$, likely due to root broadcast bottleneck or the error of measurement.

Table 7 summarizes the speedup of each stage for the large dataset from a numerical perspective. The dictionary-parallel strategy contributes most significantly to *Construct Dict* and *Build BK-Tree*, whose execution times decrease approximately $1/p$ trend. This results from both effective workload distribution and improved cache locality as the dictionary slice becomes smaller. In contrast, *Load dict* and *Get misspelled* also exhibit scalability but only reach about $1/2$ to $1/3$ of ideal linear speedup. The *Load dict* phase is constrained by the shared file system bandwidth while *Get misspelled* is primarily bounded by workload imbalance, which arise from load imbalance and uneven candidate search costs in the BK-tree. This means some words produce substantially larger candidate sets than others. To mitigate this issue, we experimented with dynamic scheduling strategies (`schedule(guided, 64)` and `schedule(dynamic, 128)`), but the performance improvement was marginal, indicating that the imbalance originates from inherent data variability rather than thread scheduling inefficiency.

Table 7. Large dataset speedup per stage relative to $p=1$ (times in ms)

Stage	T_1	T_8	$S_8=T_1/T_8$	T_{32}	$S_{32}=T_1/T_{32}$
Load dict	78	18	4.33	9	8.67
Build WordList	43	60	0.72	52	0.83
Construct Dict	469	28	16.75	4	117.25
Build BK-Tree	4,605	386	11.93	68	67.72
Get misspelled	1,935,313	455,227	4.25	88,956	21.76
Save output	12	11	1.09	11	1.09
TOTAL	1,940,523	455,733	4.26	89,103	21.78

In summary, these results demonstrate the effectiveness of parallelization when the dictionary size increases while the word list remains fixed. To further reduce the relative cost of the word list phase, utilizing a pipelined or streaming word batch would be the next step. The current implementation executes stages sequentially, which means each phase starts only after the previous one completes. However, allowing partial word list loading and candidate generation concurrently could substantially improve performance. According to [1], the overlap communication and computation is proved effective in non-collective operations. In this study, the author further suggests that to achieve significant overlap, systems should be equipped with low-latency, high-bandwidth interconnects, such as InfiniBand, and MPI implementations supporting asynchronous progress, either through progress threads or hardware-assisted communication engines.

Upper Bound. From a theoretical side, Amdahl’s Law explains the observed scaling behaviors. Table 5 shows the serial fractions for the small, medium, and large datasets are 0.0463, 0.0199, and 0.0172, respectively. These values indicate that the proportion of parallelizable work increases as the dataset size grows, which aligns with the interpretation in paragraph 4.2. In practice, the measured 32-process speedups correspond to 64%, 38%, and 38% of its theoretical speedup limits, implying medium and large datasets still have significant headroom for optimization. However, as shown in Figure 2 and Figure 3, the measured data points follow the theoretical curve closely and this upper bound ignores the parallelization overhead, such as processes startup and communication costs, which grow with the processes number, and therefore would overestimate the achievable speedup.

Bent’s Law further considers the span in algorithm and suggests the scalability limit is:

$$T_p = \Theta\left(\frac{W}{p} + T_\infty\right)$$

, where W is work and T_∞ is span. For the large dataset, $W = T_1 = 1,940,523$ and the estimated serial fraction $f = 0.0172$ give $T_\infty \approx f \times T_1 = 33,376$. Therefore,

$$T_{32} = \Theta\left(\frac{W}{32} + T_\infty\right) = 94,017 \quad (\text{ms})$$

The observed time of 89,103 is slightly below this bound, which is explainable because the bound consist hidden constant factor and both f and T_∞ are estimated by the experimental results.

Memory. Figure 7 shows the trade-off between total memory usage and execution time for different datasets and process counts. In general, increasing process number reduces execution time but increases total memory usage approximately linearly. This trend reflects that each process holds a local copy of its word list when more processes are add to process dictionary in parallel.

The reported peak memory is estimated by the summation of each maximum RSS of all ranks. This approach could slightly overestimate the actual memory usage since the peak of individual memory might occur not concurrently.

4.3 Limitation

This report mainly focuses on experiments that vary the number of processes since the effect of thread parallelism is proved limited. The minimal impact of threads is largely due to the characteristics of the BK-tree search, which visits many nodes and is inevitably highly memory bounded. As a result, adding more threads quickly occupy the available memory bandwidth and not improving computational throughput. Another factor is the uneven distribution of work during candidate generation creates data skew across threads. Even when using dynamic or guided scheduling, this imbalance cannot be fully resolved.

Besides, all evaluations were conducted on a single Spartan node. Cross-node factors, such as interconnect latency, network contention, and topology effects were not considered. Therefore, the multi-node performance should be predicted with more care.

5 Conclusion

The two-phase design and dictionary slicing show advantages for multiple processes. At $np=32$, the overall speedups reach about $26\times$ and $22\times$ for medium and large datasets, respectively. Partitioning by stage, the speedup demonstrates nearly linear scaling in dictionary-related phases, such as construct Dict and build BK-tree. These results confirm that the parallel technique is effective when the dictionary component dominates the workload. Therefore, it would be valuable to perform additional experiments that keep the same word list with different dictionary size to better understand whether the current implementation primarily accelerates dictionary-related operations or also improves candidate generation performance significantly.

Moreover, the speedup of $np=32$ in get misspelled phase is 22 times more than $np=1$, further suggests that separating the presence check and candidate generation phases successfully reduces redundant computation. Nonetheless, this step is still the main contributor to execution time. As a result, future work would focus on exploring alternative indexing structures, such as Trie approach, to deal with the memory bottleneck and investigating overlapping communication and computation to reduce overall execution time.

References

- [1] Torsten Hoeftler, Andrew Lumsdaine, and Wolfgang Rehm. 2007. Implementation and Performance Analysis of Non-Blocking Collective Operations for MPI. In *Proceedings of the 2007 International Conference on High Performance Computing, Networking, Storage and Analysis, SC07* (Reno, USA). IEEE Computer Society/ACM.

A Appendix

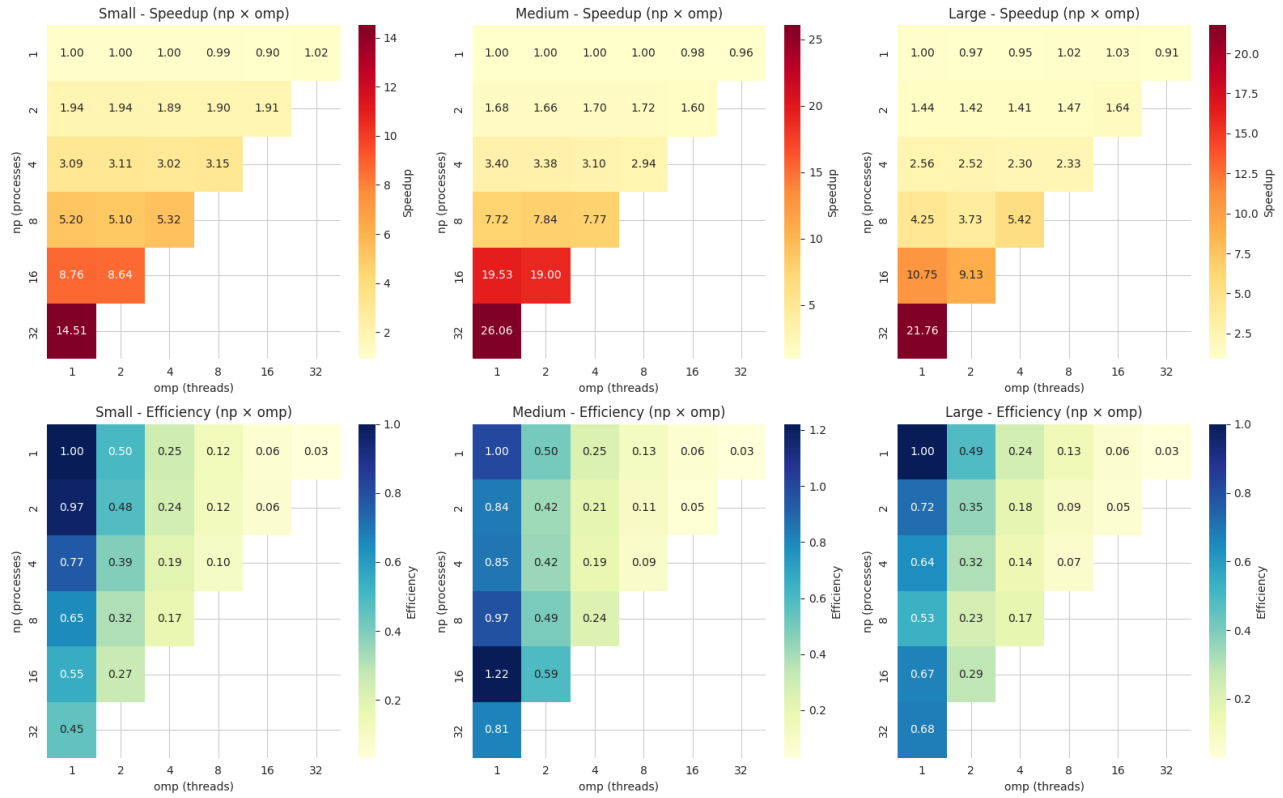


Figure 8. Speed and Efficiency